Vectorization – 27th March

Friday, 27 March 2026 6:04 PM

- **Vectorization**

English $\xrightarrow{\text{translator}}$ math.

Text → Numbers → Model → Prediction.

movie is good. → [0, 1, 1, 0, 0] → numerical representation → model understand.

↳ vector.

* Core Concept.

Step-1 Build vocabulary.

Suppose

I love ML.
I love AI.

Vocabulary → set of unique words.

→ [I, love, ML, AI].

Step-2 Convert sentence into vector.

I love ML → [1, 1, 1, 0],
I love AI → [1, 1, 0, 1]

Each position corresponds to a word.

- Type of **Vectorization**
  - Bag of words. (BoW) → Counts.
  - TF-IDF → Importance.
  - Word2Vec. → Context based meaning.
  - AverageWeVec → Sentence-level meaning.

Key Idea.

Vectorization = mapping text → numerical feature space.

↳ It not about just converting to numbers

↳ it is about meaningful representation

Poor vectorization = Poor model performance

Advantages.

- Make text usable for ML model
- Simple techniques are fast (BoW, TF-IDF).
- Work well for basic problems.

Disadvantages.

- Loose word order.
- High dimensional vector.
- Doesn't capture the meaning properly.

- **Bag of Words (BoW)**

Sentiment Analysis problem

Input

This movie is amazing.
This movie is terrible.

amazing → +ve.
terrible → -ve.

Bag. of word → we only care regarding word frequency.

Sentence → [I love machine learning] → Bag.

- Order will be lost
- Context is lost.
- Only count remains.

Step-1 : Collect all sentence.

Eg. I love ML
I love AI
AI is amazing.

Step-2 : Create vocabulary.

[I love ML, AI, is, amazing].

Step-3 : Convert each sentence into vector. → we only count the occurrence

Sentence	I	love	ML	AI	is	amazing.
I love ML	1	1	1	0	0	0
I love AI	1	1	0	1	0	0
AI is amazing	0	0	0	1	1	1

BoW = Frequency based representation of text.

Advantage

- Simple and Fast
- Easy to implement
- Works well for baseline models.
- Interpretable

Disadvantage

- Ignore word order
- Ignore meaning.
- huge dimensional vector
- Sparse matrix.

- **TF-IDF** (Term frequency and Inverse document frequency).

In simple terms., In TF-IDF, we reduce the importance of common words and highlight important ones!

Core Idea

common word → less importance
Rare but meaningful words → more important

TF-IDF helps us to answer → How important is a word in a document relative to all documents?

- How often a word appears in a document.

TF = No. of times word appear in a document

Total words in documents.

Eg.

Sentence AI AI ML.

TF of AI - $\frac{2}{3}$.

- Inverse Document frequency.

$$IDF = \log\left\{\frac{\text{Total no. of documents}}{\text{No. of document containing the word}}\right\}.$$

So, If a word appears in all document IDF → low.
If a word appears in few document IDF – high.

Final Importance score

TF-IDF = TF × IDF.

High TF + High IDF → Important word.
High TF + Low IDF → common word (ignore).

Search engine
Search ranking.
Document similarity.
Information retrieval etc.

Advantages
- Reduce impact of common words.
- Highlight important words.
- Better than BoW for many task.
- Work well for search & ranking.

Disadvantages
- Still ignore word order.
- No semantic meaning.
- Sparse vector.
- Cannot capture context.

- Word2vec

Suppose
I love deep learning.
I enjoy neural network.

King – man + woman = queen.

In real life.
words that appear in similar context → have similar meaning.

Eg. dog → bark, pet, animal.
cat → meow, pet, animal.

Core Idea of word2vec
Tell me the context → we tell you the meaning.
Instead of counting the words → we learn relationship b/w words.

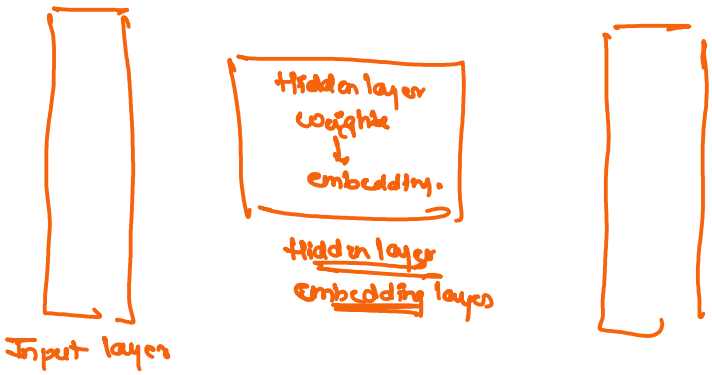
Step-1 : Representing words as a vector

Each word → dense vector (not space).

King - [0.25, -0.10, 0.78 ...]
queen [0.20, -0.12, 0.80 ...].

word embedding.
100-300 dimensions

Word2vec → shallow neural network.



Similar words will have similar vectors.

Learn from context

In word2vec there are two architecture
- CBOW. (continuous Bag of words)
- Skip-grams.

- **CBOW** → predict a word from context

I love ——→ learning.
└→ machine / Deep.

- Skip-grams → predict context from a word.

learning.

BOW.

Advantages
- Capture semantic meaning.
- Dense vector. (efficient).
- work well for similarity task.
- Learn relationship automatically.

Disadvantages
- Context independent
(same word = same vector).
- Needs larger data.
- Cannot hand poly semy. words
- Static embedding.

Word2vec

machine → T T

machine → [illegible]

learning → [illegible]

is → [illegible]

powerful → [illegible]

Sentence vector = $\frac{v_1 + v_2 + v_3 + v_4}{n}$